PROJECT PROPOSAL · LLMS AND APPLICATIONS

PipeGuard

Pre-merge detection of silent data bugs in PySpark pipeline changes, combining code, intent and data profiles

Ben Shakarof · Solo project

Week 5 · Proposal presentation

1 · MOTIVATING USE CASE

The job succeeded. The numbers are wrong.

Background

Data pipelines (Spark, Airflow) feed dashboards, reports and ML models, and change every week, more and more with AI coding assistants. Users: data engineers reviewing pull requests.

Why it matters

A silent bug does not crash the job; it writes wrong numbers. Duplicated revenue or dropped customers can feed business decisions until someone notices.

Why it is hard

No error, no log. Whether a change is a bug depends on the data (is the key unique? are there NULLs?) and on the intent (was the drop in rows on purpose?).

How it is solved today

Code review, which does not see the data; data tests, which catch only what someone thought to check; data-diff tools, which show what changed but leave a human to judge whether it is a bug.

WHAT A SILENT BUG LOOKS LIKE

One table name changed. Revenue more than doubled.

```
commit: "Use customer history table for richer attributes"
-  .join(customers,      on="customer_id", how="left")
+  .join(customers_hist, on="customer_id", how="left")
```

Output profile	Before	After
Job status	SUCCESS	SUCCESS
Rows	10,000	23,140
Distinct order_id	10,000	10,000
sum(amount)	1.20M	2.77M

VERDICT

JOIN_FANOUT

WHY

customers_hist keeps one row per customer version (SCD Type 2), so every order is duplicated. Rows rose, distinct orders did not.

Illustrative example.

1 · WHY THIS IS HARD

Not a log-reading problem

	Pipeline failure logs	Silent data bugs (this project)
Job status	Fails	Succeeds
Evidence	The error message names the cause	No error; evidence is split across code, data and intent
Read it and know?	Often: the explicit error points to the cause within seconds	No: the same diff is correct or a bug depending on the data
Existing AI tools	Failure diagnosis in AWS Glue, Databricks, Astronomer	Data-diff tools show differences; a human judges bug vs. intended

Why AI coding assistants miss it: they usually review code without representative data, so they cannot see that customers_hist has many rows per customer.

Sources: AWS Big Data Blog, generative AI troubleshooting for Spark in AWS Glue (2024); Databricks docs, AI_DIAGNOSE; Astronomer blog, Otto (2026).

2 · PROJECT TASK

Silent bug or not, and which one?

Problem statement

Multi-class classification of a pipeline change into one of 8 labels: 6 silent-bug types and 2 OK types. Scope: one semantic change per example; both versions run successfully.

Input → Output

Input: code diff, commit message, and the output profiles of the old and new versions run on the same test data.

Output: label with confidence.

Proposed contribution

A reproducible benchmark for classifying silent bugs introduced by PySpark code changes, from code diffs, developer intent and before/after output profiles.

Validated labels. Bugs are injected, and a scenario is kept only if the expected behavior appears in the output.

Research question. Do runtime data profiles improve over code-only baselines?

Closest work: Chen, Phani & Schelter, PrismaDV (arXiv 2604.21765) tests errors in incoming data, not code changes. Novelty to be checked in the interim review.

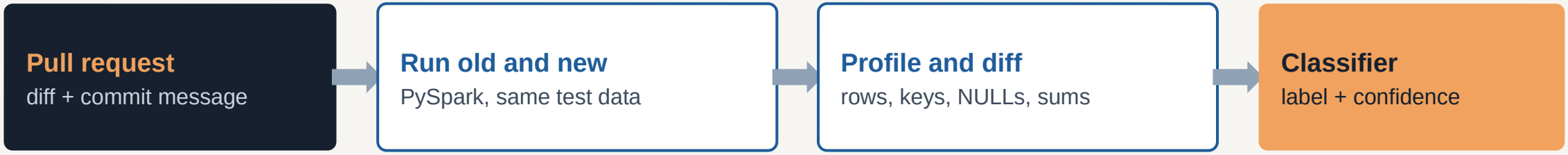
2 · OUTPUT LABEL SET

Six silent-bug types, two OK types

Label	Code clue	Data signature
JOIN_FANOUT	join table or key changed	rows up, distinct keys unchanged
JOIN_DROP	left join became inner	rows down, unmatched keys vanish
UNION_MISALIGN	union instead of unionByName	two columns swap distributions
NULL_FILTER_DROP	!= or NOT IN filter	more rows dropped than matched; NULL% to 0
TIMEZONE_SHIFT	date conversion or timezone setting	totals equal, daily split shifts
CAST_TRUNCATION	cast to a narrower type	values rounded, sums drop
OK_REFACTOR	code restructured	profile identical
OK_INTENDED	logic changed on purpose	change matches the explicit change request

Labeling rule: one semantic change per example. OK_INTENDED comes from an explicit change request; the commit message is a supporting signal, not proof.

Run both versions, profile, then classify



Off-the-shelf

- **Primary:** GPT-4o-mini few-shot + CoT on code + profile
- Rule baseline: one rule per bug type, plus majority class
- GPT-4o-mini on code only (ablation)

Fine-tuned

- **Primary:** CodeBERT + classification head (HF Trainer)
- Sentence embeddings + LR head (light comparison)
- Stretch: small LLM with LoRA

Fair comparison: GPT and the learned models see the same information (commit + diff + profile diff) in each ablation (code / profile / both); rules use the profile numbers · long inputs truncated to 512 tokens by a fixed policy

4 · DATA SPECIFICATION AND GENERATION

Validated scenarios plus a held-out test

D1 · BASE

Pipelines and test data

8 clean PySpark pipelines written by me (orders, customers, events). Synthetic input tables with controlled traps: non-unique keys, NULLs, events near midnight.

D2 · TRAIN + TEST

Change scenarios

6 bug operators, refactors and intended changes. Commit messages by GPT-4o-mini from the change request only, with no label hints.

Label: injected type, kept only if validated. Target ~1,500.

D3 · TEST ONLY

LLM-generated changes

~40 change requests done by an LLM on the controlled pipelines; bugs found against a reference solution. A first generalization check, not a per-class benchmark.

Label: me + a second annotator on 20 cases; Cohen's κ.

Flow: assign pipelines to train / test groups → generate scenarios within each group (attribute grid: bug type × data trap × message style) → run both versions → assert expected behavior → reject invalid cases. Every label in every split; D3 separate and test only; sizes are targets.

5 · METRICS AND KPIS

Measure the end result and every step

End-to-end

- Detection F1: bug vs. OK (primary)
- Macro-F1 over the 8 labels
- False-alarm rate on intended changes
- Confusion matrix

Per step

- Validation pass rate of injected bugs
- Commit messages: realism and no label hints, on a sample
- D3 labels: Cohen's κ
- Inference cost and latency per check

Protocol

- Leave-pipelines-out split
- Ground truth: validated injection (D2), human labels (D3)
- 3 seeds, mean ± std, plus per-pipeline results
- Error analysis; D3 with raw counts

Success criteria (targets, on the held-out D2 test): code + profile beats the matched code-only model by ≥ 5 F1 points, with ≤ 10% false alarms on intended changes. D3 is reported separately.

PLAN · SOLO PROJECT

About one working day per week

Week	Milestone
6-7	Pipelines, data generator, bug operators, validation harness
8	Dataset, EDA; rule and GPT baselines
9	Interim: data, EDA, baselines, 3 papers
10-11	CodeBERT, input ablation, D3 test set
12	Error analysis, figures; Airflow and CI demo
13	Final presentation; repo two weeks later

MUST

Validated dataset, rule and GPT baselines, evaluation on unseen pipelines (fallback: 4 pipelines)

SHOULD

CodeBERT, D3 test set, full ablation, Airflow DAG and GitHub Actions demo

STRETCH

LoRA, more pipelines and bug types